

# Overview

## Navigation

zsync is a file transfer program. It allows you to download a file from a remote server, where you have a copy of an older version of the file on your computer already. zsync downloads only the new parts of the file. It uses the same algorithm as [rsync](#). However, where rsync is designed for synchronising data from one computer to another within an organisation, zsync is designed for file distribution, with one file on a server to be distributed to thousands of downloaders. zsync requires no special server software — just a web server to host the files — and imposes no extra load on the server, making it ideal for large scale file distribution.

- [Home](#)
- [Download](#)
- [Client use](#)
- [Server use](#)
- [Technical Paper](#)
- [Mailing List](#)
- [Credits & Contact](#)

zsync is [open source](#), distributed under version 2 of the Artistic License. Feedback, bugs reports and patches are welcome.

## Advantages of zsync

zsync fills a gap in the technology available for large-scale file distribution. Three key points explain why zsync provides a genuinely new technique for file distribution:

- **Client-side rsync** — zsync uses the rsync algorithm, but runs it on the client side, thus avoiding the high server load associated with rsync.
- **Rsync over HTTP** — zsync provides transfers that are nearly as efficient as rsync -z or cvsup, without the need to run a special server application. All that is needed is an HTTP/1.1-compliant web server. So it works through firewalls and on shared hosting accounts, and gives less security worries.
- **Handling for compressed files** — rsync is ineffective on compressed files, unless they are compressed with a patched version of gzip. zsync has special handling for gzipped files, which enables update transfers of files which are distributed in compressed form.

The special handling of compressed files is, as far as I know, entirely new and unique to zsync. The combination of client-side rsync and HTTP is also unique to zsync, to the best of my knowledge.

I have put up [technical papers](#). There is a current paper, which reflects zsync's latest development, and which I update with new figures as I run new tests and comparisons; there are also older snapshots of the technical paper, included for reference and citation.

## Practical Engineering

zsync is only useful if people offer zsync downloads. I have made a deliberate effort to make zsync simple and reliable in use, and easy to deploy.

- **Single meta-file** — zsync downloads are offered by building a .zsync file, which contains the meta-data needed by zsync. This file contains the precalculated checksums for the rsync algorithm; it is generated on the server, once, and is then used by any number of downloaders.
- **Easy file format** — .zsync files have a simple text header, so they can be edited with any good editor to change details like the download URL.
- **Flexible use** — The .zsync contains either a relative URL to the full content of the file, so you can either generate and offer it on the same server as hosts the full download (and so it can be distributed like any other file onto mirror sites). Or it can contain one or more absolute URLs (the client will try multiple URLs if necessary to find the content), meaning the .zsync can be hosted on a different server entirely; in fact, the .zsync can be generated and offered by a third party, while still leaving most of the download to the original distribution site.

Because zsync is a file distribution method, it is useful only when people offer .zsyzns for downloads. It's a little hard to demonstrate the program and build up a user base because of this. So I am suggesting a number of uses for zsync on the [client page](#); these include some zsync downloads that I am offering downloads myself to give some immediate use to the program.

If you are a server operator looking to use zsync for file distribution, see the [server page](#), which gives some guidance on how to create and offer .zsync files.

## Current Status

I am fairly happy with how zsync works so far, but I am still open to new ideas and optimisations. I have made sure that the format is fairly extensible in a compatible way; I expect that users of the current version of zsync will not experience compatibility problems with later versions for the foreseeable future.

I am also looking to freeze the interface between the zsync library and the current command-line client; I will at some point split zsync into a dynamic library, which implements the format and allows it to be embedded into other programs, and the client program. This to some extent depends on whether anyone is interested in embedding zsync into other programs; if I don't hear of anyone intending to do so, this might wait until I have a use for it myself.

I am interested in [hearing feedback about zsync](#). Bugs, crashes, and feature suggestions are welcome. Perhaps most of all I am interested in hearing about uses of zsync: I would like to hear about any serious use of zsync, because knowing how people are using it will affect which features I work on. If there are features that you need to enable a particular use, then I would like to hear about that too, particularly anyone that is interested in using zsync as a library as part of a larger program; please note that support for multiple files has already been asked about many times, but due to lack of time I am not likely to work on this in the near future.

---



[Privacy notice](#)